Multi-Bottleneck Fairness on RDMA Fabrics: An Empirical Study of Deployable Congestion Control and an RCP Service Mode over In-Network Telemetry

Haoyu Wang*, Bo Sheng*, Zerin Shaima Meem†, Xiaoqian Zhang†
*University of Massachusetts Boston, Boston, MA, USA {haoyu.wang001, bo.sheng}@umb.edu
†University of Nebraska Omaha, Omaha, NE, USA {zmeem, xiaoqianzhang}@unomaha.edu

***Abstract*—Modern datacenter traffic increasingly crosses several contended links in series, yet deployable RDMA congestion control is evaluated almost exclusively on single-bottleneck benchmarks. We present an empirical study of multi-bottleneck fairness on lossless RDMA fabrics. On a parameterized parking-lot benchmark with a fluid max-min oracle, *every* deployable scheme we test is unfair to the multi-bottleneck flow — DCQCN 1.7–$2.3\times$, TIMELY 1.8–$2.0\times$, DCTCP 1.6–$1.8\times$, HPCC 1.9–$2.0\times$ — growing with bottleneck count and robust to start-time jitter. For HPCC we localize the mechanism: a winner-take-all equilibrium in which the multi-bottleneck flow collapses to ~ 0.4 Gbps. Six sender-only corrections fail, including a sender-side mirror of RCP driven by the same INT signals — evidence that the obstacle is not information but *consistency*: max-min needs one shared reference rate per link, and private per-sender estimates never agree. Placing that reference at the switch — RDMA-RCP, an RCP service mode carried in one 64-bit INT field with one scalar of state per port — restores the max-min allocation (penalty $\approx 1.005\times$) with zero PFC, rate-only or with a canonical $cwnd = R \cdot RTT$ window, and cuts allreduce-style coflow completion time by $\sim 21\%$ at every ECMP seed. We also measure the deployment boundary: RCP-mode and HPCC traffic do not coexist fairly on a shared link — even under a weighted-DRR reservation, because the classes' control signals remain port-global — so per-class deployment needs sliced telemetry or a separate fabric. Stock HPCC remains byte-for-byte unchanged alongside.**

***Index Terms*—datacenter networks, congestion control, RDMA, in-network telemetry, max-min fairness, RCP**

I. Introduction

HPCC [1] and its derivatives represent the state of the art in datacenter RDMA congestion control: the switch stamps raw per-hop state into INT headers, and the sender computes a precise utilization estimate toward a target η. On the canonical single-bottleneck benchmarks HPCC matches or beats DCQCN, TIMELY, and DCTCP on flow completion time and queueing — and that is where deployable RDMA congestion control is, almost exclusively, evaluated.

Modern datacenter traffic, however, increasingly crosses *several contended links in series*: cross-pod ML collectives, disaggregated memory and storage, multi-tenant aggregation and core layers. The practical stake is concrete: a collective completes when its *slowest* flow completes, and the slowest flows are precisely the cross-fabric, multi-bottleneck ones. On a 16-flow ring allreduce over a k=4 fat-tree, we measure stock HPCC inflating job completion time by $\sim 27\%$ over what max-min allocation delivers — at every ECMP hash seed we test (§V-D). Whether deployable RDMA congestion control is fair to such traffic has not been carefully measured. This paper measures it.

We proceed with a deterministic, oracle-grounded methodology. Our congestion-control stack is a refactored re-port of the original HPCC ns-3 simulator onto a tagged ns-3.45 release; before drawing any conclusion from it we validated the re-port by reproducing the original paper's baseline comparisons across five schemes and two production workloads (§V), and the full reproduction record and raw results are public. Our findings:

1) **The failure is endemic, and we characterize HPCC's instance of it** (§II, §V-C). Every deployable scheme we test is unfair to the multi-bottleneck flow — DCQCN 1.70–$2.32\times$, TIMELY 1.83–$2.01\times$, DCTCP 1.61–$1.78\times$, HPCC 1.90–$1.96\times$ at $N = 2$–4 — all growing with bottleneck count, and robust to start-time jitter. For HPCC we isolate the mechanism: the failure is *winner-take-all* (the long flow collapses to ~ 0.4 Gbps against ~ 23 Gbps competitors; $3.5\times$ for small flows), RTT-independent, immune to HPCC's own per-hop multi-rate mode and to INT compression (PINT), and compounded past four bottlenecks by a wire-format cap ($\texttt{maxHop} = 5$) that corrupts the control input. A single-bottleneck control (N=1: 1.10–$1.39\times$ pairwise spread, direction set by arrival order) separates HPCC's baseline rate lottery from the systematic multi-bottleneck penalty on top.
2) **Sender-side control cannot close the gap — and the reason is consistency, not information** (§III). We evaluate six sender-only designs: five heuristics, plus the strongest possible variant — a full sender-side mirror of the RCP recursion driven by the same INT-visible inputs the switch would use. The mirror still fails ($1.56\times$ even with synchronized starts; $1.65\times$/$0.81\times$ under staggered arrival, the *direction* set by arrival order), while the identical law at the switch holds $\approx 1.0\times$. Max-min

requires all flows at a link to adopt *one shared reference rate*; private multiplicative estimates preserve their initial disagreement. We present this as controlled empirical evidence, not a formal impossibility.

3) **An RCP service mode over INT restores max-min, verified end to end** (§IV, §V). RDMA-RCP carries a switch-computed RCP fair rate in one 64-bit INT field with one scalar of state per egress port, selected per traffic class alongside byte-for-byte unchanged stock HPCC. It reaches penalty $\approx 1.005\times$ across bottleneck counts, asymmetric sizes, staggered and jittered starts, and ECMP hash seeds, with zero PFC — operating rate-only or with a canonical RCP window ($cwnd = R \cdot \mathrm{baseRTT}$), and cutting allreduce coflow JCT by $\sim 21\%$ at every seed. We equally report its deployment boundary: sharing a link with stock HPCC serves neither class — even under a weighted-DRR reservation, because both controllers still read port-global telemetry (§IV-C); per-class deployment needs sliced signals or a separate fabric. The mechanism is RCP — we claim the measurement study and the placement result, not a new control law.

Artifacts. The implementation, the topology/workload generators, the max-min oracle, and deterministic scripts that regenerate the figures (`make_figures.py`) and the ablation/sensitivity/baseline tables (`run_sensitivity.py`) from source are available in our artifact-review repository.[1]

II. Background and Motivation

A. HPCC, briefly

HPCC's per-hop utilization estimate for hop i is

$$u_i = \frac{txRate_i}{C_i} + \frac{\min(qlen_i, qlen_i^{\mathrm{prev}}) \cdot maxRate}{C_i \cdot W} \qquad (1)$$

where the first term reflects bandwidth utilization and the second is a queue-drain term that keeps standing queues bounded. In single-rate mode the sender takes $U = \max_i u_i$, smooths it over the base RTT, and applies

$$new\_rate = \begin{cases} curRate / \max c + R_{AI} & \max c \geq 1 \\ curRate + R_{AI} & \text{otherwise} \end{cases}$$

with $\max c = U/\eta$. The window W is sized to $m_{win} \cdot rate/NIC\_line$ under `var_win`.

B. The multi-bottleneck parking lot

The *parking lot* is a classic congestion-control fairness benchmark [2], [3]. The name is an analogy: a long boulevard (the long flow's path) is crossed by a series of side streets, and at each intersection a different car (a single-link cross flow) merges on and exits. Because every link is equally contended, the *max-min fair* allocation [2] gives every flow an equal share, so the long flow should run exactly as fast as each cross flow despite traversing many contention points rather than one. The parking lot therefore isolates the multi-bottleneck fairness question in its simplest form, with no confounding asymmetry in link capacity or hop count of the competitors. One control is essential before reading any deviation as a multi-bottleneck effect: on a *single* shared bottleneck (N=1) with two symmetric 50 MB flows, stock HPCC already allocates unequally (37.20 vs 26.72 ms, a $1.39\times$ spread), while RDMA-RCP equalizes them exactly (33.89 vs 33.89 ms). Stock HPCC therefore carries a baseline rate unfairness between symmetric flows; the multi-bottleneck penalty we characterize is the *additional* deviation on top of it, growing to 1.90–$1.96\times$ at $N = 2$–4.

We construct a parameterized parking-lot benchmark with N inter-switch bottleneck links (25 Gbps each) and 100 Gbps host links (Figure 1). One "long" flow traverses all N bottlenecks; one "cross" flow traverses each single bottleneck. Flows are 50 MB by default and start simultaneously at $t = 0$. With two flows sharing each 25 Gbps link, the max-min fair rate is 12.5 Gbps for every flow. All experiments are deterministic: each configuration produces byte-identical artifacts across repeated runs (verified).

Metric. We compare each flow's actual flow completion time (FCT) against an idealized **fluid max-min oracle** — a continuous bandwidth-sharing model with no protocol dynamics (no slow start, no queueing, no convergence delay) in which the bandwidth is allocated max-min fairly at every instant. We compute the allocation by *progressive filling* [2]: all active flows raise their rate in lockstep until some link saturates; the flows crossing that link are frozen at their current rate; and the process repeats on the remaining flows until every flow is frozen. This is the textbook algorithm for max-min fair rates. Because flows complete and depart over time, we run it *event-driven* — recomputing the allocation at each flow completion — so the oracle FCT correctly accounts for the bandwidth freed as short flows finish. The oracle is therefore the best FCT any rate-allocation scheme could achieve under max-min fairness, and is exact for our fluid topologies. The headline metric is the **unfairness ratio** $\max_{\text{path}} FCT/\mathrm{mean}(\text{short-path FCT})$; the oracle's value is 1.0 for equal-size, simultaneous flows.

C. Findings F1–F4: characterizing the gap

Magnitude. Stock HPCC's unfairness ratio at $N \in \{2, 3, 4\}$ is 1.90, 1.92, 1.96: the multi-bottleneck flow's FCT is roughly twice that of single-bottleneck competitors. **There is a non-monotonic regime change at** $N \geq 5$, where the long flow becomes favored ($0.51\times$); this turns out to be an INT-overflow artifact at `maxHop` $= 5$ and we exclude $N \geq 5$ from the rest of this paper's stock comparisons (Figure 2).[2]

Robustness. Size and start-time perturbations give 1.81–$3.53\times$ (Figure 4), worst ($3.5\times$) when the multi-bottleneck flow is small — the latency-sensitive RPC case. *Multi-rate HPCC* (per-hop EWMA, $\min_i Rc_i$) helps but does not fix it: $1.44/1.50/1.75\times$ at $N = 2/3/4$, still growing with N. *RTT-equalization rules*

[1] https://github.com/UNOCourseDemo/hpcc-fs

[2] The 5-hop cap is HPCC's original wire format, not an artifact of our re-port: the reference simulator hard-codes `maxHop` $= 5$ in its INT header [4], consistent with the paper's 42-byte worst-case INT overhead (five 8-byte hop records). Conversely, a parking-lot path at $N \leq 4$ traverses at most five switches — *within* the INT budget — so the retained $N = 2$–4 measurements are structurally free of the overflow.

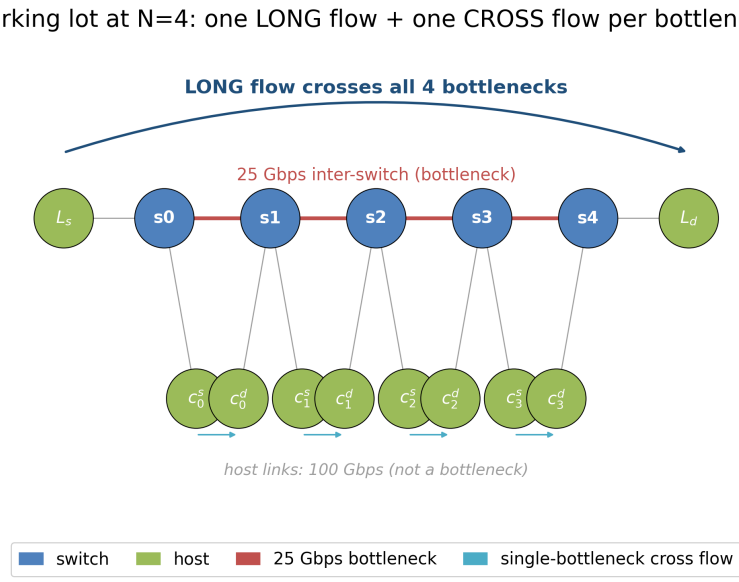

Fig. 1. Parking-lot benchmark at $N = 4$. The long flow $L_s \to L_d$ crosses every bottleneck; each cross flow $c_i^s \to c_i^d$ traverses exactly one. Host links are not bottlenecks.

out RTT bias: the penalty stays $\sim 1.9\times$ across a $4.5\times$ sweep of cross-flow RTTs, including cross flows with up to $1.8\times$ the long flow's RTT — this is not RTT unfairness.

Trace diagnosis. At $N = 4$ (Figure 5 left) the long flow collapses to ~ 0.38 Gbps while the four cross flows hold ~ 23.2 Gbps each for the entire 19 ms contention window, with zero PFC and zero loss — *winner-take-all*: at every shared link the local single-bottleneck flow wins and the long flow is squeezed.

III. The Limits of Sender-Side Control

A natural first design instinct — and the one our project explored exhaustively — is to keep all state and computation at the sender, since HPCC's value proposition is its stateless switch design. We evaluated six sender-only correction variants behind a new `mb_mode` config knob, on top of unchanged HPCC; the `mb_mode = 0` path is byte-identical to stock HPCC (verified). Five are small heuristic changes to `UpdateRateHp`; the sixth is the strongest design the endpoint admits — a full sender-side mirror of the RCP recursion.

A. Five heuristic variants

1) **k-aware additive increase** (`mb_mode 1`). The flow counts congested hops k from INT and scales R_{AI} by $\gamma \cdot k$. Result at $N = 4$: penalty drops from 1.96 to $1.63\times$ at $\gamma = 50$, still nowhere near 1.0.
2) **Debiased max/mean blend** (`mb_mode 2`). $U_{\text{eff}} = (1-\lambda)\max u_i + \lambda \text{mean}_{u_i \geq \eta} u_i$. Result: penalty barely moves ($1.95\times$ at $\lambda = 1$), revealing that per-hop u_i are nearly equal in our setting — there is no significant max-bias to debias.
3) **Responsibility-weighted MD** (`mb_mode 3`). The flow's multiplicative decrease is scaled by its measured share $own\_rate/txRate_{\text{btl}}$ at the bottleneck. Result: $1.89\times$ — almost no effect, because the MD branch barely fires at the η-hold equilibrium.
4) **k-th-root MD** (`mb_mode 4`). $new\_rate = curRate \cdot (1/\max c)^{1/k}$. Result: $1.92\times$.
5) **Global share-aware additive increase** (`mb_mode 5`). Applied to *every* flow: $R_{AI}^{\text{eff}} = 2(1 - share)\gamma R_{AI}$ when congested. Result: $1.71\times$ at $\gamma = 20$.

TABLE I
Sender-side ablation: best penalty at $N = 4$ for each variant.

mb_mode	mechanism	best penalty
0	stock HPCC	$1.96\times$
1	k-aware AI ($\gamma = 50$)	$1.63\times$
2	debiased blend ($\lambda = 1$)	$1.95\times$
3	responsibility-weighted MD	$1.89\times$
4	k-th-root MD	$1.92\times$
5	global share-aware AI ($\gamma = 20$)	$1.71\times$

B. The strongest sender-side design: a virtual-RCP mirror

The five variants above are heuristics, and a fair objection is that they do not exhaust the design space. The strongest sender-side design is implied by §IV itself: RCP's inputs — capacity C, aggregate load y, queue q — are all sender-visible in HPCC's INT record. So let the sender run the switch's recursion *privately*: per hop, maintain a virtual fair rate $\hat{R}_i$ updated by the identical RCP rule from the flow's own INT samples, and send at $\min_i \hat{R}_i$ (`mb_mode 6`; same gains, same $R_0 = C/2$, the sender's own base RTT as control interval — the settings most favorable to the endpoint). If the obstacle were *information*, this variant would match the switch.

It does not (Table II). With synchronized starts it reaches only $1.56\times$; under staggered arrival it lands at $1.65\times$ or

TABLE II
SENDER-SIDE VIRTUAL RCP (`MB_MODE 6`) VS. THE SWITCH MODE: SAME CONTROL LAW, SAME INT-VISIBLE INPUTS, DIFFERENT PLACEMENT. N=4 PARKING LOT; "LATE" = THAT SIDE STARTS +5 MS AFTER THE OTHER.

	synchronized	long late	crosses late
switch (shared per-port R)	$1.005\times$	$1.008\times$	$1.003\times$
sender (private $\hat{R}$)	$1.560\times$	$1.654\times$	$0.813\times$

$0.81\times$ — the *direction* of the unfairness set by arrival order — while the identical law at the switch holds 1.003–$1.008\times$ in all three scenarios. The mechanism is visible in the update itself: the recursion is multiplicative, so senders driven by a common observed (y, q) sequence multiply their private estimates by (nearly) the same factor, and the *ratio* between their estimates never contracts. A late-arriving flow initializes fresh while incumbents hold evolved state, and no later observation reconciles them; RTT heterogeneity alone — the synchronized case — already breaks the common trajectory.

C. The structural reason: consistency, not information

Two observations explain the pattern. *First, the η-hold equilibrium:* once the contended links settle at target utilization η, HPCC's multiplicative term $1/\max c \approx 1$ and every flow holds its current rate — whoever grabbed more at startup keeps it. Additive nudges (modes 1, 5) are orders of magnitude too slow relative to flow lifetimes (our 200 ms runs are $\sim 10^4$ RTTs and they never converge), and MD-side modifications (modes 3, 4) rarely fire at the hold point.

Second — what mode 6 isolates — max-min requires a shared reference. The information needed to compute the fair share is sender-visible; what the sender cannot obtain is *agreement*. The max-min allocation is reached when every flow at a bottleneck adopts the *same* rate. Private per-sender estimates, updated multiplicatively from asynchronous samples, preserve their initial disagreement instead of contracting it; a per-port register at the switch is the cheapest consistent shared reference — every packet reads the same R, so a newcomer adopts the incumbents' rate within one RTT. This is why RCP [5], [6] and XCP [3] place the computation in the network. Our contribution is the controlled demonstration that, for INT-style RDMA control, the *placement* is what matters: six sender-side designs spanning heuristics to a full RCP mirror fail, while the identical law at the switch succeeds. We present this as strong empirical evidence, not a formal impossibility result.

IV. RDMA-RCP DESIGN

The empirical and structural arguments of §III motivate a design that (a) makes the *dominant* flows yield rather than merely nudging the victims, and (b) places a fair-rate *signal* at the switch, since the sender cannot recover the fair share from a load signal alone. We do not have the switch count flows and divide; instead each port runs a standard RCP control law on aggregate load and queue (below) whose fixed point *is* the max-min fair rate C/N — the switch converges to the fair share without ever materializing N. We take the most minimal mechanism consistent with these constraints, packaged as an *operating mode* selected per traffic class alongside stock HPCC on the same INT transport (§IV-C).

Background: RCP The Rate Control Protocol (RCP) [5]–[7] is a router-assisted congestion control in which each link advertises a *single* rate R to *all* flows crossing it, holding no per-flow state. The key point is that R is not an aggregate budget for the link; it is the rate *each individual flow* should send at, such that when all of them do, the link is exactly full and shared equally. A router raises R when its link has spare capacity and lowers it when a queue forms; routers stamp R into passing packets, each flow sends at the minimum R along its path, and that rate is echoed to the source. A single flow adopting R therefore does not seize the link — it takes its share, because R has already been driven down by the load the other flows place on the same port. Concretely, with N flows each sending R the offered load is NR, and the controller drives $NR \to C$ (idle link $\Rightarrow$ R rises; standing queue $\Rightarrow$ R falls), so at equilibrium $R \approx C/N$ — the max-min (processor-sharing) fair share, tracked *without ever counting* N: as flows arrive or leave, R self-adjusts (a lone flow's R rises to C; a third arrival drops everyone to $C/3$). RCP thus supplies exactly the missing ingredient from §III: a fair-share signal computed where the flow count is implicitly observable (the link), not at the sender. RDMA-RCP runs this control law once per egress port and carries its R in the INT header; a flow's path-minimum R approximates network-wide max-min fairness (within 1.2% of the oracle on our benchmarks), since a flow bottlenecked elsewhere contributes less load here and the loop hands the slack to the rest.

A. The mechanism

The entire protocol is Algorithms 1 and 2. The switch maintains one scalar fair rate per egress port and stamps the path-minimum into a single header field; the sender reads that field from the returning ACK and adopts it. No per-flow switch state, no per-hop record, no receiver changes.

At the switch (Alg. 1). Each egress port keeps a single scalar fair rate R_j, refreshed once per control interval d — a fabric-wide constant set to the maximum base RTT, $21.4\,\mu$s in all our experiments — by the standard RCP rule $R \leftarrow R\,(1 + (\alpha(C - y) - \beta q/d)/C)$, where y is the load observed over the last interval and q the queue backlog. The term $\alpha(C - y)$ chases spare capacity (positive while the link is under-utilized, pushing R up) and $\beta q/d$ brakes when a standing queue forms (pulling R down); they balance at $y \approx C$, $q \approx 0$, which is the $R \approx C/N$ fixed point above. We use the RCP defaults $\alpha = 0.4$, $\beta = 0.226$ (untuned; the result is insensitive to them, §V-H). The port's entire state is three numbers — R_j, the last update time, and a byte-counter snapshot — with *no* per-flow data.

On the wire. The FS INT mode (`IntHeader::FS`) carries one 64-bit field, `fairRate`; the serialized cost is 8 bytes per packet *independent of hop count* (vs. 8 bytes *per hop* for stock HPCC). Each switch min-aggregates R_j into this field (first

Algorithm 1 RDMA-RCP switch — per egress port j

State R_j (fair rate), t_j (last update), B_j (txByte snapshot)
Const C_j link capacity; α=0.4, β=0.226; d control interval ($\approx$ RTT); $R_{\min}$
procedure ONDEQUEUE(pkt, j) ▷ every packet leaving port j
    if R_j uninitialised **then**
        $R_j \leftarrow C_j/2$; $t_j \leftarrow now$; $B_j \leftarrow txBytes_j$ ▷ conservative start
    end if
    if $now - t_j \geq d$ **then** ▷ one control interval elapsed
        $y \leftarrow (txBytes_j - B_j) \cdot 8 \,/\, (now - t_j)$ ▷ observed load
        $q \leftarrow queueBytes_j \cdot 8$ ▷ backlog (bits)
        $R_j \leftarrow R_j \cdot \big(1 + (\alpha(C_j - y) - \beta\, q/d)/C_j\big)$ ▷ RCP update
        $R_j \leftarrow \mathrm{clamp}(R_j,\, R_{\min},\, C_j)$
        $t_j \leftarrow now$; $B_j \leftarrow txBytes_j$
    end if
    if $pkt.fairRate = 0$ **then** ▷ first hop on the path
        $pkt.fairRate \leftarrow R_j$
    else
        $pkt.fairRate \leftarrow \min(pkt.fairRate,\, R_j)$ ▷ path-min
    end if
    $txBytes_j \leftarrow txBytes_j + \mathrm{sizeof}(pkt)$
end procedure

Algorithm 2 RDMA-RCP sender (receiver echoes the header unchanged)

on QP-create: $rate \leftarrow R_{\min}$ ▷ conservative start

procedure ONACK(a)
    if $a.fairRate = 0$ **then return** ▷ no signal yet
    end if
    $rate \leftarrow \mathrm{clamp}(a.fairRate,\, R_{\min},\, NICrate)$ ▷ adopt path-min fair share
end procedure

hop initialises it), so the receiver sees the path-minimum fair rate and echoes the header back unchanged — no receiver changes.

At the sender (Alg. 2). The new ACK handler `HandleAckFs` reads `fairRate` from the returning ACK and sets the sending rate to it, clamped to $[R_{\min}, NICrate]$. That single assignment is the whole sender control loop.

B. Startup smoothing

Two cold-start details: the switch initializes $R = C/2$ (not C, else the first stamp is line rate), and the `cc_mode 11` sender starts at `m_minRate` so the first RTT does not blast before any ACK returns. A flow sees a path-complete fair-rate signal in one RTT; full convergence takes several control intervals (§V: within 5% of fair share by $\sim$0.6 ms, $\sim$30 RTTs at our 21 μs base RTT; within 1% by 2.3 ms) — fast and path-length-independent, though not literally one-RTT.

C. Additive integration and coexistence

RDMA-RCP is `cc_mode == 11`. The mode-3 (HPCC) and mode-10 (PINT) wire formats and switch/sender code paths are completely untouched; we add a fourth case to (a) `IntHeader::Mode`, (b) its serialization, (c) the switch's mode-dispatch, (d) the sender's mode-dispatch, and (e) one branch in the validation driver. The driver also forces the QP window to 0 in FS mode (rate-only control), since `var_win` sizes the window from the NIC line rate rather than the bottleneck path BDP and would otherwise cap the multi-hop flow's effective window well below what is needed at its fair rate. In the shipped `cc_mode 11` the INT wire format is a global choice, so a fabric runs one mode at a time. To test the per-class deployment the framing suggests, we added `cc_mode 12`: a wire layout identical to stock HPCC's 42-byte INT record (so stock packets stay byte-for-byte unchanged), with the switch and sender dispatching *per packet* on the traffic class. The FS class reuses the record's first eight bytes for the fair rate and never pushes hop records; it therefore pays stock's header overhead, which is conservative for our claim. Two regression gates hold: with every flow in the stock class, `cc_mode 12` reproduces `cc_mode 3`'s FCTs *exactly*; with every flow in the FS class it reproduces `cc_mode 11`'s 1.005$\times$ penalty at zero PFC.

Coexistence is not free (negative result). On the N=4 parking lot with the long flow in the RCP class and the cross flows in the stock class, *neither class gets its fair share*: rate-only RCP overshoots (21.6 Gbps against a 12.5 fair share, squeezing stock to 11.1; penalty 0.51$\times$ — the starvation reverses), the windowed variant under-shoots (6.2 Gbps, 2.53$\times$), and a separate priority queue does not help (0.51$\times$; HPCC keeps its queue near zero and forfeits round-robin turns). The cause is this paper's theme inverted: RCP's fixed point $R = C/N$ presumes every flow on the port obeys R — non-compliant flows are invisible to the inference exactly as flow count is invisible to a sender.

We then tested the obvious remedy: a weighted-DRR egress scheduler (additive `dwrr_weights`; byte-deficit DRR over the non-priority queues) reserving each class its exact 50% per-link entitlement. *It changes nothing* (0.511$\times$): the scheduler never binds, because stock HPCC's INT still reports *port-global* bytes and queue — the stock class sees $u \approx 1$ from RCP traffic, holds low, and never offers its reserved share, which RCP then absorbs. A fair scheduler cannot help a class whose controller will not offer its share; coexistence requires the reservation to extend into the *telemetry* — per-class C, y, q — i.e. true slicing, or a separate fabric. All variants ship with the artifact (`run_mixed_mode.py`); PFC is zero throughout.

V. EVALUATION

A. Methodology

All experiments use the deterministic parking-lot benchmark of §II, driven by our `hpcc-validation.cc`. Flows are 50

TABLE III
METRICS USED IN THIS PAPER.

unfairness ratio	long-flow FCT / mean cross-flow FCT (oracle = 1.0)
oracle-norm. penalty	per-flow FCT / fluid max-min oracle FCT
raw long/short ratio	mean long FCT / mean short FCT (ECMP study)
slowdown	FCT / same-size standalone FCT
coflow JCT	max FCT over a coflow's flows

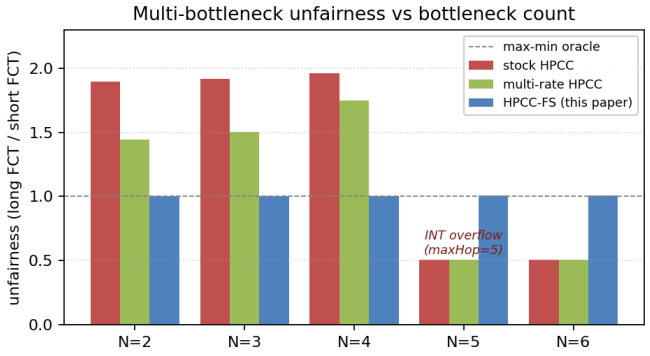

Fig. 2. Multi-bottleneck unfairness vs bottleneck count N. RDMA-RCP is flat ≈ 1.0 across the entire sweep; stock and multi-rate HPCC fail at $N \leq 4$ and break at $N \geq 5$ (INT-overflow regime).

MB unless noted; sim time is 200 ms; the optimized binary is used throughout. We compare against stock HPCC (`cc_mode 3`, `multi_rate = false`), stock HPCC multi-rate, HPCC-PINT, and the `mb_mode` ablation suite. The simulator is a refactored re-port of the original HPCC ns-3 stack onto a tagged ns-3.45 release; before using it here we verified that the re-port still reproduces the original paper's baseline comparisons (HPCC vs. DCQCN, TIMELY, DCTCP, and PINT across WebSearch and FB-Hadoop workloads at 30–70% load on a 320-server fat-tree). The full reproduction record — per-run configurations, results, and analysis — ships with the artifact,[3] and its verdict is that the re-port matches the original paper's central findings. Table III defines every metric used below.

B. Headline: N-sweep

Stock and multi-rate HPCC are starved across $N = 2$–4 (penalty 1.90–1.96× for stock, 1.44–1.75× for multi-rate), then break entirely at $N \geq 5$, where the per-hop INT record overflows the `maxHop` cap (the apparent 0.51× there is the overflow artifact, not a real allocation). RDMA-RCP is flat at 1.003–1.007× across the entire sweep — hop-count-independent by construction, since its single fair-rate field does not grow with path length. HPCC-PINT, which carries one compressed utilization field, inherits HPCC's load-based control and so still suffers the same ∼1.8× (1.77–1.88×) multi-bottleneck penalty — confirming that the fix requires a fair-share signal, not merely a smaller INT footprint. All RDMA-RCP runs have zero PFC events.

[3]Reproduction record and methodology: https://github.com/UNOCourseDemo/hpcc-fs/tree/main/examples/hpcc/repro-verification. The raw per-run results (554 MB compressed) are archived at https://1drv.ms/f/c/6052297178cce52b/IgAsSQ7gNfdhQrNejod27CpkAYQs8xZs4QtGc_KKVWZjQLs.

TABLE IV
UNFAIRNESS RATIO AND PFC PAUSES ON THE PARKING LOT: EVERY DEPLOYABLE SCHEME FAILS MULTI-BOTTLENECK MAX-MIN; ALL WORSEN WITH N.

scheme	N=2	N=3	N=4	PFC
DCQCN	1.70×	1.90×	2.32×	6–34
TIMELY	1.83×	1.96×	2.01×	72–156
DCTCP	1.61×	1.72×	1.78×	256–280
HPCC	1.90×	1.92×	1.96×	0
RDMA-RCP	**1.003**×	**1.003**×	**1.005**×	0

C. The failure is endemic: deployable baselines

Is multi-bottleneck unfairness HPCC-specific? We ran every deployable RDMA scheme in our stack on the same benchmark, with harness parity to the original HPCC evaluation (window-based schemes keep their window; DCQCN, TIMELY, and DCTCP run windowless exactly as in their reference configurations). Table IV: *every* scheme is unfair to the multi-bottleneck flow and worsens with bottleneck count — DCQCN worst at N=4 (2.32×). HPCC is the norm, not an outlier; only the RCP mode reaches max-min, and only HPCC and the RCP mode hold zero PFC.

D. When max-min is the objective: allreduce-style coflow JCT

The workload class that motivates multi-bottleneck fairness is collective communication: a training step's shard exchange or allreduce completes only when its *slowest* flow completes, so the job completion time is the coflow's maximum FCT — and the slowest flows are precisely the cross-pod, multi-bottleneck ones that stock HPCC starves. We run a 16-flow ring allreduce chunk-exchange over all hosts of the k=4 fat-tree (4 inter-pod, 4 intra-pod-different-edge, 8 same-edge segments; 50 MB each; JCT = max FCT), across the same five ECMP hash seeds. RDMA-RCP cuts JCT by 19.5–22.7% (mean 21.4%) at *every* seed, and makes it nearly placement-invariant (17.9–18.0 ms across seeds, versus stock's 22.3–23.3 ms) — equalized flows cannot be victimized by an unlucky hash. Adding eight 2 MB background flows leaves the JCT gain intact (+20.6% mean) and, in this mix, speeds the background flows up ∼3× (0.9–1.3 ms vs 2.2–4.3 ms): under winner-take-all dynamics whether a short flow wins or loses is a placement lottery (they lose here, they won in the WebSearch mix of §V-M), and max-min removes the lottery. Read as a coflow, the 8-flow ECMP workload of §V shows the same effect: makespan improves 9.8–26.5% across seeds.

E. Structurally different topologies

Tree fabric (3-tier, no ECMP). A 3-tier tree with unique shortest paths (15 nodes; 25 Gbps inter-switch, 100 Gbps host; Figure 3). Stock penalty 1.36×, RDMA-RCP **1.003**×, PFC = 0.

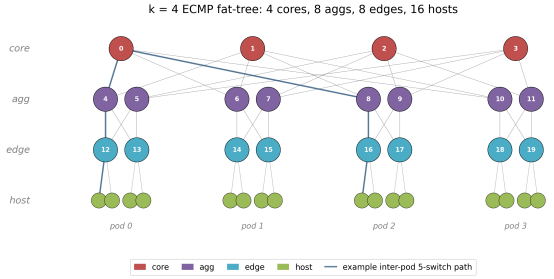

Fig. 3. The $k = 4$ ECMP fat-tree (4 cores, 8 aggs, 8 edges, 16 hosts in 4 pods); the highlighted line is a representative inter-pod 5-switch path. The tree-fabric topology (unique shortest paths, no ECMP) is described in the text.

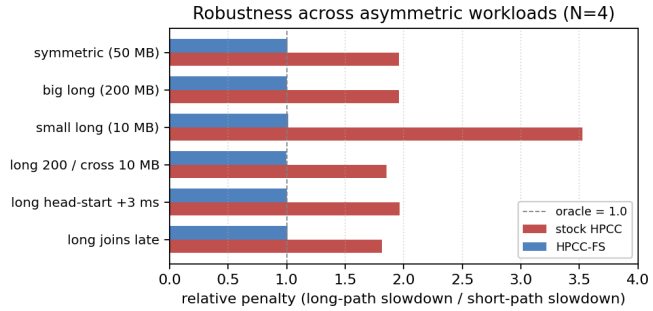

Fig. 4. Robustness across asymmetric-size and staggered-start variants at $N = 4$. RDMA-RCP is within 1.0 ± 0.012 across all six scenarios, including the small-long worst case where stock reaches $3.53\times$.

$k = 4$ **fat-tree (with ECMP).** The canonical fat-tree (20 switches + 16 hosts; 48 links; Figure 3) with the simulator's hash-based ECMP routing. Workload: 4 inter-pod flows (5-switch paths) plus 4 intra-pod different-edge flows (3-switch paths). We report two distinct quantities. (i) The *oracle-normalized penalty* — mean long-path slowdown divided by mean short-path slowdown, each measured against the max-min oracle — which removes the effect of ECMP load placement: stock $1.34\times$, RDMA-RCP $\mathbf{1.001}\times$ (every contended flow lands at its oracle FCT). (ii) The *raw* long/short FCT ratio, which under RDMA-RCP is still $\sim 1.32\times$ — but this residual is *not* CC unfairness: ECMP hashes some flows onto under-loaded cores so they finish early, inflating the raw ratio independently of the congestion controller. PFC $= 0$ for both schemes. The point is that, once ECMP load placement is normalized away, RDMA-RCP removes the multi-bottleneck penalty; the path-min fair-rate aggregation is path-invariant, so ECMP requires no design changes. *Hash-seed sensitivity.* Because a single deterministic run fixes one placement, we re-ran the workload across five ECMP hash seeds (an additive `ecmp_seed_offset` knob shifts every switch's hash seed; the default preserves stock behavior), reporting the *mean*-long/mean-short raw ratio throughout this paragraph. Under stock HPCC that ratio varies 0.97–$1.34\times$ across seeds *with no change to the congestion controller* — direct evidence that the raw spread is placement-dominated. At the seed where placement produces real contention it drops $1.338\times \rightarrow 1.001\times$ under RDMA-RCP. RDMA-RCP is at or below stock at four of five seeds; at seed 202 neither scheme is contended (both ratios <1, i.e. the long flows are *favored* by placement) and the two agree within 1.6%. The oracle-normalized conclusion is therefore seed-robust (`run_ecmp_seeds.py` in the artifact).

F. Robustness across asymmetric workloads

Across six perturbations (Figure 4), RDMA-RCP holds within 1.0 ± 0.012. The worst stock case (small multi-bottleneck flow, $3.53\times$ penalty) drops to $1.012\times$.

Start-time jitter. Two ensembles rule out synchronization artifacts of deterministic simultaneous starts. Jittering all five flows' start times by $U[0, 1]$ ms (five deterministic seeds), stock HPCC's penalty stays 1.96–$1.99\times$ and RDMA-RCP stays 1.008–$1.017\times$. On the $N{=}1$ control, the pairwise spread persists at 1.10–$1.39\times$ across offsets from $10\,\mu$s to 5 ms with its *direction* flipping by arrival order — the η-hold rate lottery, not a synchronization artifact — while RDMA-RCP equalizes the pair exactly at every offset.

TABLE V
WINDOW ABLATION: THE NIC-SCALED WINDOW BREAKS FAIRNESS; A FAIR-RATE-SCALED CANONICAL RCP WINDOW PRESERVES IT (ALL ZERO PFC).

N	rate-only (default)	NIC-scaled window	$cwnd = R \cdot \mathrm{RTT}$
2	$\mathbf{1.003}\times$	$1.50\times$	$\mathbf{1.004}\times$
3	$\mathbf{1.003}\times$	$2.00\times$	$\mathbf{1.004}\times$
4	$\mathbf{1.005}\times$	$2.49\times$	$\mathbf{1.006}\times$

G. Ablation: windows must scale with the fair rate

RDMA-RCP defaults to rate-only operation — the per-flow window cap is disabled (§IV). Table V shows why, and shows the constraint is narrower than "no windows": re-enabling HPCC's `var_win` window degrades the penalty to 1.5–$2.5\times$, worsening with path length, because that window is sized from the NIC line rate rather than the smaller multi-hop bottleneck path. The problem is the window's *scale*, not windowing itself: a canonical-RCP endpoint that sizes its window from the adopted fair rate, $cwnd = R \cdot \mathrm{baseRTT}$ (the `fs_rcp_window` knob), matches rate-only fairness while bounding in-flight data per flow. Deployments wanting a hard in-flight cap can keep one; it must simply track R, not the NIC.

H. Parameter sensitivity

We use the standard RCP defaults ($\alpha = 0.4$, $\beta = 0.226$) untouched. Sweeping each parameter one at a time at $N = 4$ — $\alpha \in [0.2, 0.6]$, $\beta \in [0.1, 0.4]$, startup $R_0/C \in [0.25, 1.0]$, $minRate \in [100, 2000]$ Mb/s — the penalty stays within $[1.004, 1.008]\times$ with zero PFC throughout (even $R_0 = C$: the `m_minRate` sender start absorbs the first interval). Within this benchmark family the RCP parameters are not load-bearing; we do not claim the same for arbitrary topologies or joint sweeps.

TABLE VI
HPCC'S HOME TURF (SINGLE-BOTTLENECK INCAST). RDMA-RCP HOLDS FAR LESS QUEUE THAN HPCC AT EVERY STARTUP, AND A MODERATE STARTUP ALSO MATCHES OR BEATS HPCC'S FCT. THE SAME STARTUP LEAVES THE MULTI-BOTTLENECK FAIRNESS HEADLINE UNCHANGED (1.003–1.004×, PFC = 0).

	mean FCT	tail FCT	peak queue
HPCC	258.6 μs	459.8 μs	143 KB
RDMA-RCP (startup 1 Gbps)	297.5 μs	490.3 μs	**64 KB**
RDMA-RCP (startup 5 Gbps)	271.2 μs	464.3 μs	**49 KB**
RDMA-RCP (startup 8 Gbps)	**242.0 μs**	**416.8 μs**	**38 KB**

I. Queueing and PFC

With the smoothed startup (§IV-B), RDMA-RCP produces **zero PFC events** on every workload in the N-sweep and the robustness matrix. ECN marking remains; queue length is bounded (peak $\approx$46 KB at $N = 4$, versus $\approx$118 KB for stock HPCC on the same workload); no flow loss.

J. Do-no-harm

The stock `cc_mode 3` smoke tests and N-sweep are byte-identical to the pre-change baseline: the RDMA-RCP additions (implementation name `HPCC-FS`, `cc_mode 11`) only add code; no existing function's output for existing inputs is altered.

K. HPCC's home turf: competitive FCT, lower queue

What does the RCP mode cost on *single*-bottleneck workloads — HPCC's home turf? We measure the validated incast benchmark (3 senders $\rightarrow$ 1 receiver over 25 Gbps; 12 flows, 100–400 KB), sweeping the sender's startup rate:

Two things stand out (Table VI). First, RDMA-RCP holds *far less queue* than HPCC at *every* startup (38–64 KB vs 143 KB) — RCP's $\beta q/d$ term drains queue aggressively. Second, the only place HPCC leads is FCT at the conservative 1 Gbps startup ($\sim$15%); raising the startup closes and then *reverses* that gap — at 8 Gbps RDMA-RCP beats HPCC's mean FCT (242 vs 259 μs) at roughly one-quarter the queue, with zero PFC. The startup is a clean knob (queue rises again only past the point where the aggregate startup rate exceeds link capacity), and crucially it does *not* disturb the multi-bottleneck headline: at 8 Gbps the parking-lot penalty stays 1.003–1.004× ($N = 2, 3, 4$) with robustness variants 1.002–1.009×, all PFC = 0 (a second operating point; the headline runs use the 1 Gbps startup). So on the home-turf workloads we measured, RDMA-RCP's fairness mode comes at *little-to-no FCT cost and a queue benefit* — not a trade-off. Worst-case in-flight data is also bounded in rate-only operation: before the first fair-rate ACK a sender emits at most *startup* $\times$ baseRTT (1 Gbps $\times$ 21.4 μs $\approx$ 2.7 KB per sender; even 1000 synchronized senders inject $\sim$2.7 MB in the first RTT), and the $cwnd = R \cdot \mathrm{RTT}$ variant (§V-G) restores a hard per-flow cap thereafter.

We are careful not to overclaim: RDMA-RCP still lacks HPCC's per-hop, window-bounded reaction, so on rapidly-varying or adversarial-microburst workloads (which we did not evaluate) HPCC's precision may matter; recovering that while keeping the fairness mode is what the hybrid overlay of §VII targets.

L. Per-flow rate trace, before vs after

The before-vs-after rate trace (Figure 5) is the cleanest qualitative result. Stock HPCC: the long flow collapses to $\sim$0.4 Gbps while the four cross flows hold $\sim$23 Gbps each for the full 19 ms contention; RDMA-RCP: all five flows converge to $\sim$12.5 Gbps within $\sim$4 ms and stay locked at fair share.

M. Scope of the headline claim

The $\approx$1.0× headline is a statement about the *congestion-control component* of multi-bottleneck unfairness on contention benchmarks — not a claim of universal FCT improvement on production workloads. Four first-class limits. (i) *Mixed short-flow workloads* do not exhibit the penalty we target: under a WebSearch distribution stock HPCC's oracle-normalized penalty is already 1.001× (RDMA-RCP: 0.581×), and on mean slowdown RDMA-RCP trades $\sim$15% off multi-bottleneck flows for $\sim$45% on short paths (a mean dominated by sub-10 KB flows). Short-flow-FCT workloads [8], [9] may prefer stock HPCC; max-min matters when multi-bottleneck flows gate a job (§V-D) — a per-class mode selection, not a global replacement. (ii) *Fabric scale*: at $k = 6, 8$ our scaling workload does not form persistent inter-pod contention (both schemes' penalty <1), so the mode is exercised, but untested, where large-fabric contention would arise; a saturating workload is future work. (iii) *ECMP placement*: operators still observe a raw long/short spread ($\sim$1.3× at the contended placement) from hash load placement; RDMA-RCP removes the CC component, not the placement component — a routing problem. (iv) *Coexistence*: the two modes do not share a link fairly — even under a weighted-DRR reservation, since both controllers still read port-global signals (§IV-C); per-class deployment needs sliced telemetry or a separate fabric.

VI. RELATED WORK

Datacenter transport. DCTCP [10], DCQCN [11], TIMELY [12], and Swift [13] trade buffering for latency via ECN- and delay-based control; Homa [8] and pFabric [9] prioritize short flows; HPCC [1] drives control from per-hop INT, with On-Ramp [14] and PowerTCP [15] extending the precise-signal philosophy. Across this line the target is single-bottleneck FCT, tail latency, or queueing; multi-bottleneck fairness receives little attention — and §V-C shows every deployable member of the family fails it.

In-network telemetry. INT [16] lets the data plane stamp per-hop state into packets on programmable pipelines, and HPCC was among the first to drive congestion control directly from it. The cost is header overhead, which later work minimizes: HPCC-PINT [17] compresses the per-hop record into a single probabilistic field, and Poseidon [18] restructures INT-based control for deployability on commodity hardware. RDMA-RCP continues this minimal-telemetry direction but changes *what* the field carries: not a raw load sample to be interpreted

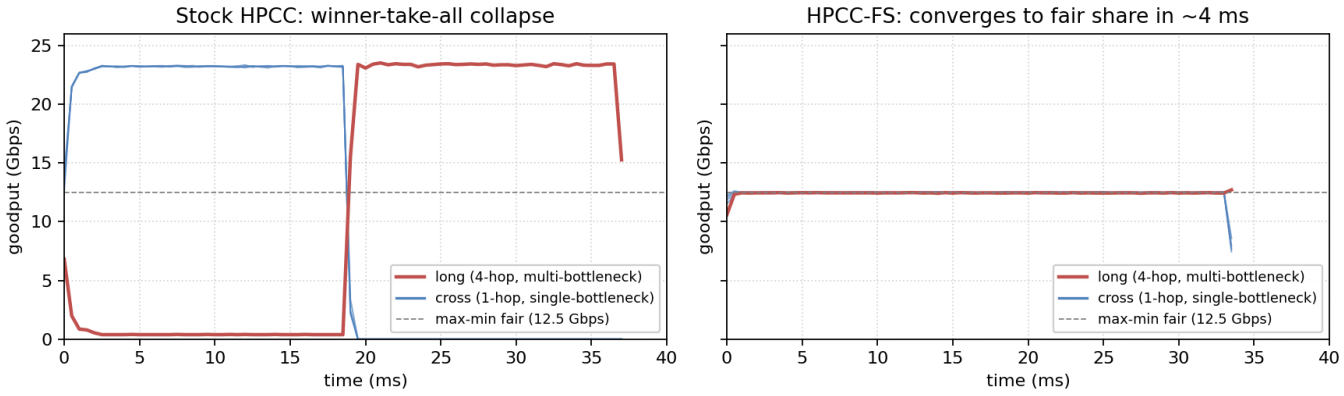

Fig. 5. Per-flow goodput at $N = 4$, stock HPCC (left) vs RDMA-RCP (right). Stock collapses the long flow to $\sim$0.4 Gbps while cross flows hold $\sim$23 Gbps for 18 ms (winner-take-all). RDMA-RCP converges all five flows to within 5% of the max-min fair share (12.5 Gbps) in $\sim$0.6 ms and to within 1% by $\sim$2.3 ms.

by the sender, but an explicit fair *rate* computed at the switch and min-aggregated along the path—one 64-bit field, like PINT, yet a decision rather than a measurement.

Explicit-rate and switch-assisted control. Having the switch compute and return a sending rate is a classic idea. ATM ABR explicit-rate schemes such as ERICA [19] computed a fair share per port decades ago; XCP [3] decomposed switch feedback into separate efficiency and fairness controllers; and RCP [6] reduced this to a single fair-rate scalar per link with no per-flow state, packet-stamped, with senders taking the path minimum. RCP is the closest mechanism to ours: RDMA-RCP adopts its update rule but situates it as a *minimal fairness signal injected into an existing INT load-signaling system* rather than a standalone protocol, keeping HPCC's single-rate sender and per-hop transport. RDMA-RCP's per-port update and path-min aggregation *are* RCP's; the endpoint differs (rate-only by default, though the canonical window $cwnd = R \cdot \mathrm{RTT}$ performs equally, Table V), the control interval is a fabric constant, updates are event-driven, and the port applies $R_0 = C/2$ and a rate floor. What RDMA-RCP adds over deploying RCP as a separate protocol is packaging, not control: one field on the INT transport the datacenter already runs, a byte-identical stock HPCC path, and per-`cc_mode` selection — no second protocol stack. ABC [20] argues, for wireless links, that switches should compute the right control signal rather than expose raw load; we apply the same philosophy with minimal departure from HPCC's datacenter design.

In-switch fairness. Fair queueing [21], DRR [22], CSFQ [23], and AFD [24] enforce fairness by per-packet queueing or dropping (CSFQ/AFD in the same stateless-per-flow spirit as RCP); programmable schedulers [25], [26] realize such policies at line rate. RDMA-RCP instead *advertises* one fair rate per egress port and lets rate-controlled senders converge to it — one scalar of egress state, no per-flow tracking, no extra queues. Match-action pipelines [27], [28] make its per-port computation (an EWMA, a multiply–add, one stamped field) implementable on the switches HPCC's INT already assumes.

Closely related switch-assisted DC schemes. Bolt [29] accelerates HPCC's loop with sub-RTT per-flow switch assistance, targeting tail latency and ramp-up — orthogonal to our objective: its feedback carries load/supply, not a fair share, so the shared- reference argument of §III applies to its senders too; a faster load-signal loop converges faster to the same equilibrium. A head-to-head would measure reaction speed, not the placement question this paper isolates; we leave it to future work.

Fairness foundations. Max-min fairness, progressive filling, and the parking lot are classical [2]. Our contribution is not a new fairness definition but the measurement: six sender-side corrections — including a full RCP mirror — fail to realize this classical objective from sender-visible INT, while one switch-computed rate achieves it.

VII. Discussion

Relation to HPCC, and a hybrid path. RDMA-RCP demonstrates *where* the fair-share signal must live; it inherits RCP's character — fair, low-queue — without HPCC's aggressive single-flow precision (§V-K). A natural future hybrid runs HPCC unchanged and applies the switch's fair rate only as a per-flow *ceiling*, $\min(rate_{\mathrm{HPCC}}, fairRate)$; the open obstacle is HPCC's NIC-sized window, which can hold a victim below fair even after the ceiling frees bandwidth (§V-G suggests a fair-rate-scaled window as the fix).

INT hop limit. Stock HPCC's per-hop INT cost grows with path length and its reference format caps at `maxHop` $= 5$, corrupting the control input beyond four bottlenecks; RDMA-RCP's one-field format is hop-count-independent and stays at $\approx 1.0\times$ at $N = 5, 6$.

Scaling, heterogeneity, hardware. At $k = 6, 8$ the fabric's $(k/2)^2$ cores spread inter-pod flows so persistent contention does not materialize for our scaling workload (both schemes' penalty <1; RDMA-RCP adds no overhead, 0 PFC) — a saturating workload is future work. Our bottlenecks are uniformly 25 Gbps; RCP handles heterogeneous speeds gracefully

in the literature, but we have not verified it here. The path-min aggregation is path-invariant under ECMP (§V); residual FCT spread is load placement, a routing issue. In hardware, the per-port update is three multiplies and an add per control interval, and the per-packet min-aggregation is a 64-bit compare-and-write — well within programmable-switch budgets.

Limitations. Three topology families; in-sim baselines only (DCQCN, TIMELY, DCTCP, multi-rate HPCC, PINT) — no Bolt/PowerTCP re-implementation and no testbed; per-class coexistence needs a reserved share we do not design (§IV-C); the ECMP study is a small controlled workload (though seed-swept), not a realistic mixed-size + ECMP combination. All numbers come from one (re-ported) simulator: the re-port reproduces the original paper's baselines end-to-end and the $N \leq 4$ regime sits within the 5-hop INT budget by construction, but an independent-implementation cross-check remains future work. We did, however, verify that the result is insensitive to the RCP parameters (§V-H), so the absence of tuning is not a hidden confound.

VIII. Conclusion

We have shown that HPCC's load-signaling INT design has a structural fairness failure in multi-bottleneck workloads — a real, robust, RTT-independent winner-take-all collapse with $\sim 2\times$ FCT penalty for the multi-bottleneck flow, up to $3.5\times$ for small such flows. No sender-side correction we attempted (across five representative variants) closes the gap, because at the η-hold equilibrium additive nudges are too slow and the sender cannot compute its fair share without knowing the link's flow count. A minimal mechanism that suffices on our benchmarks is **one fair-rate field** in a new INT mode plus **one scalar of state per switch port** (and, in our prototype, disabling the per-flow window cap in FS mode — see Section IV). The resulting mode, RDMA-RCP, is offered alongside stock HPCC rather than in its place — an option for traffic that requires multi-bottleneck max-min fairness with zero PFC on the same INT fabric. It achieves max-min fairness with penalty $1.005\times$ and zero PFC at $N = 4$ across all workloads we tested, converging sub-millisecond and independent of path length, while leaving the original HPCC code path unchanged. Sharing an unreserved link with stock HPCC, however, lets the FS class capture bandwidth (§IV-C): a scheduler reservation alone does not suffice — the mode belongs on sliced telemetry or a separate fabric, not in free competition.

References

[1] Y. Li, R. Miao, H. H. Liu, Y. Zhuang, F. Feng, L. Tang, Z. Cao, M. Zhang, F. Kelly, M. Alizadeh, and M. Yu, "HPCC: High precision congestion control," in *Proc. ACM SIGCOMM*, 2019, pp. 44–58.

[2] D. Bertsekas and R. Gallager, *Data Networks*, 2nd ed., 1992.

[3] D. Katabi, M. Handley, and C. Rohrs, "Internet congestion control for future high bandwidth-delay product environments," in *Proc. ACM SIGCOMM*, 2002, pp. 89–102.

[4] Alibaba, "HPCC reference simulator (ns-3)," https://github.com/alibaba-edu/High-Precision-Congestion-Control, 2019, `int-header.h`: `static const uint32_t maxHop = 5`.

[5] N. Dukkipati, M. Kobayashi, R. Zhang-Shen, and N. McKeown, "Processor sharing flows in the internet," in *Proc. IWQoS*, 2005.

[6] N. Dukkipati, "Rate Control Protocol (RCP): Congestion control to make flows complete quickly," Ph.D. dissertation, Stanford University, 2008.

[7] N. Dukkipati and N. McKeown, "Why flow-completion time is the right metric for congestion control," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 36, no. 1, pp. 59–62, 2006.

[8] B. Montazeri, Y. Li, M. Alizadeh, and J. Ousterhout, "Homa: A receiver-driven low-latency transport protocol using network priorities," in *Proc. ACM SIGCOMM*, 2018.

[9] M. Alizadeh, S. Yang, M. Sharif, S. Katti, N. McKeown, B. Prabhakar, and S. Shenker, "pfabric: Minimal near-optimal datacenter transport," in *Proc. ACM SIGCOMM*, 2013.

[10] M. Alizadeh, A. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, and M. Sridharan, "Data center TCP (DCTCP)," in *Proc. ACM SIGCOMM*, 2010, pp. 63–74.

[11] Y. Zhu, H. Eran, D. Firestone, C. Guo, M. Lipshteyn, Y. Liron, J. Padhye, S. Raindel, M. H. Yahia, and M. Zhang, "Congestion control for large-scale RDMA deployments," in *Proc. ACM SIGCOMM*, 2015, pp. 523–536.

[12] R. Mittal, V. T. Lam, N. Dukkipati, E. Blem, H. Wassel, M. Ghobadi, A. Vahdat, Y. Wang, D. Wetherall, and D. Zats, "TIMELY: RTT-based congestion control for the datacenter," in *Proc. ACM SIGCOMM*, 2015, pp. 537–550.

[13] G. Kumar, N. Dukkipati, K. Jang, H. M. G. Wassel, X. Wu, B. Montazeri, Y. Wang, K. Springborn, C. Alfeld, M. Ryan, D. Wetherall, and A. Vahdat, "Swift: Delay is simple and effective for congestion control in the datacenter," in *Proc. ACM SIGCOMM*, 2020.

[14] S. Liu, A. Ghalayini, M. Alizadeh, B. Prabhakar, M. Rosenblum, and A. Sivaraman, "Breaking the transience-equilibrium nexus: A new approach to datacenter packet transport," in *Proc. USENIX NSDI*, 2021.

[15] V. Addanki, M. Apostolaki, M. Ghobadi, S. Schmid, and L. Vanbever, "PowerTCP: Pushing the performance limits of datacenter networks," in *Proc. USENIX NSDI*, 2022, pp. 51–70.

[16] C. Kim, A. Sivaraman, N. Katta, A. Bas, A. Dixit, and L. J. Wobker, "In-band network telemetry via programmable dataplanes," in *Proc. ACM SIGCOMM (Industrial Demo)*, 2015.

[17] L. S. Yang, R. Ben Basat, M. Mitzenmacher, M. Apostolaki, A. Klein, E. Saito, and A. Mendelson, "PINT: Probabilistic in-band network telemetry," in *Proc. ACM SIGCOMM*, 2020, pp. 662–677.

[18] W. Wang, M. Moshref, Y. Li, G. Kumar, T. S. E. Ng, N. Cardwell, and N. Dukkipati, "Poseidon: Efficient, robust, and practical datacenter CC via deployable INT," in *Proc. USENIX NSDI*, 2023.

[19] S. Kalyanaraman, R. Jain, S. Fahmy, R. Goyal, and B. Vandalore, "The ERICA switch algorithm for ABR traffic management in ATM networks," *IEEE/ACM Transactions on Networking*, vol. 8, no. 1, 2000.

[20] P. Goyal, A. Narayan, F. Cangialosi, S. Narayana, M. Alizadeh, and H. Balakrishnan, "ABC: A simple explicit congestion controller for wireless networks," in *Proc. USENIX NSDI*, 2020, pp. 353–372.

[21] A. Demers, S. Keshav, and S. Shenker, "Analysis and simulation of a fair queueing algorithm," in *Proc. ACM SIGCOMM*, 1989.

[22] M. Shreedhar and G. Varghese, "Efficient fair queuing using deficit round robin," in *Proc. ACM SIGCOMM*, 1995.

[23] I. Stoica, S. Shenker, and H. Zhang, "Core-stateless fair queueing: Achieving approximately fair bandwidth allocations in high speed networks," in *Proc. ACM SIGCOMM*, 1998.

[24] R. Pan, L. Breslau, B. Prabhakar, and S. Shenker, "Approximate fairness through differential dropping," *ACM SIGCOMM Computer Communication Review*, vol. 33, no. 2, 2003.

[25] A. Sivaraman, S. Subramanian, M. Alizadeh, S. Chole, S.-T. Chuang, A. Agrawal, H. Balakrishnan, T. Edsall, S. Katti, and N. McKeown, "Programmable packet scheduling at line rate," in *Proc. ACM SIGCOMM*, 2016.

[26] Z. Yu, C. Hu, J. Wu, X. Sun, V. Braverman, M. Chowdhury, Z. Liu, and X. Jin, "Programmable packet scheduling with a single queue," in *Proc. ACM SIGCOMM*, 2021.

[27] P. Bosshart, G. Gibb, H.-S. Kim, G. Varghese, N. McKeown, M. Izzard, F. Mujica, and M. Horowitz, "Forwarding metamorphosis: Fast programmable match-action processing in hardware for SDN," in *Proc. ACM SIGCOMM*, 2013.

[28] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, 2014.

[29] S. Arslan, Y. Li, G. Kumar, and N. Dukkipati, "Bolt: Sub-RTT congestion control for ultra-low latency," in *Proc. USENIX NSDI*, 2023.